University of Geneva

Faculty of Science

Timing Attacks

Advanced Security
14X

Michel Jean Joseph Donnet

2026-05-07

Contents

1 Introduction

The cryptography goal is to ensure the confidentiality, integrity, and authenticity of information, and it has become increasingly important in the digital age with the rise of the internet and digital communication where sensitive information is frequently transmitted. To achieve these goals, various cryptographic algorithms and protocols have been developed, including symmetric and asymmetric encryption, hashing, and digital signatures. These algorithms are based on mathematical problems that are computationally difficult to solve, such as factoring large integers or computing discrete logarithms, which provide the basis for their security since they are believed to be infeasible to break with current computational resources.

However, although these algorithms are designed to be secure against direct attacks, the implementation of these algorithms can introduce vulnerabilities that can be exploited by attackers. The vulnerabilities reside in the way the algorithms are implemented and executed, rather than in the algorithms themselves. Indeed, attackers can exploit side channels, which are unintended information leaks that occur during the execution of cryptographic algorithms such as timing information, power consumption, electromagnetic emissions, or even sound. By analyzing these side channels, attackers can gain insights into the internal workings of the cryptographic algorithm and potentially recover sensitive information such as secret keys.

In this report, we will focus on timing attacks, which are a type of side-channel attack that exploits the time it takes for a cryptographic algorithm to execute. Timing attacks can be used to recover secret keys or other sensitive information by measuring the time it takes for a cryptographic operation to complete and analyzing the variations in timing based on different inputs or conditions.

2 Timing attacks on RSA

RSA is a widely used asymmetric encryption algorithm that relies on the difficulty of factoring large integers to provide security. It allows in particular for sharing a symmetric key securely over an insecure channel, enabling secure communication between parties without the need for a pre-shared secret key.

The base principle of RSA resides in the use of a pair of keys: a public key for encryption and a private key for decryption. The RSA algorithm consists of three main steps: key generation, encryption, and decryption.

2.1 RSA basics

The key generation process involves selecting two large prime numbers, p and q, and computing their product $n = p \times q$, which serves as the modulus for both the public and private keys. The choice of p and q is crucial for the security of RSA, as the difficulty of factoring n into its prime factors is what provides the security of the algorithm. Indeed, if an attacker can factor n into p and q, they can compute the private key and break the encryption.

The algorithm of key generation can be summarized as follows:

1. Choose two distinct large prime numbers p and q.
2. Compute $n = p \times q$ and $\varphi(n) = (p-1)(q-1)$.
3. Choose an integer e such that
   - $1 < e < \varphi(n)$
   - $\gcd(e, \varphi(n)) = 1$
4. Compute d such that $e \cdot d \equiv 1 \bmod \varphi(n)$.

The public key consists of the pair (n, e), while the private key consists of the pair (n, d).

The encryption process consists of taking a plaintext message m and computing the ciphertext $c = m^e \bmod n$ using the public key, and the decryption process consists of taking the ciphertext c and computing the plaintext message $m = c^d \bmod n$ using the private key.

For convenience, we will use the following notations throughout the report:
- n: the modulus, which is the product of two large prime numbers p and q.
- e: the public exponent, which is an integer that is coprime to $\varphi(n)$.
- d: the private exponent, which is an integer such that $e \cdot d \equiv 1 \bmod \varphi(n)$.
- m: the plaintext message, which is an integer that is less than n.
- c: the ciphertext, which is an integer that is less than n.

2.2 Timing attacks on RSA

Before the exchange of symmetric keys, the client and the server need to perform an RSA encryption and decryption operation to establish a secure communication channel. The timing of these operations can be exploited by attackers to infer information about the private key.

As mentioned earlier, the security of RSA relies on the difficulty of factoring large integers, but the implementation of RSA can introduce vulnerabilities that can be exploited by attackers. The goal of the attacker is to recover the unknown private key d thanks to the knowledge of the public key (n, e) and the ability to perform encryption and decryption operations on the server using the RSA algorithm.

2.3 Square-and-multiply algorithm

Timing attacks on RSA exploit the fact that the time it takes to perform certain operations in the RSA algorithm can vary based on the input values and the internal state of the algorithm. For example, the time it takes to perform the modular exponentiation operation $c^d \bmod n$ can vary based on the value of d and the input ciphertext c. Indeed, a simple implementation of the modular exponentiation operation can use a square-and-multiply algorithm, as described in the following python code:

```python
def square_and_multiply(y, x, n):
    s = 1
    y %= n

    while x > 0:
        if (x % 2) == 1:
            s = (s * y) % n  # Extra multiplication here when x is odd !
```

```
        y = (y * y) % n
        x = x >> 1

    return s
```

This code computes $y^x \bmod n$ using the square-and-multiply algorithm, which is an efficient method for performing modular exponentiation. Indeed, computing $y^x \bmod n$ for very large values of x is computationally expensive.

The base idea of this algorithm is that computing y^x can be done by taking the binary representation of x, which is basically the decomposition of x into powers of 2, and then iteratively computing $y^x \bmod n$ by taking at each step the previous result and multiplying it by y if the current bit of x is 1, and then squaring the input y.

For instance, suppose we want to compute $6^{13} \bmod 17$.

- The binary representation of 13 is 1101, which corresponds to the powers of 2: $2^3 + 2^2 + 2^0$.
- We start with $s = 1$ and $y = 6$.
- For the first bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (1 \cdot 6) \bmod 17 = 6$
  - $y = y^2 \bmod 17 = 6^2 \bmod 17 = 2$.
- For the second bit (0), we compute
  - s remains unchanged since the bit is 0, so $s = 6$.
  - $y = y^2 \bmod 17 = 2^2 \bmod 17 = 4$. Note that here, y^2 corresponds to $y^{2^2} = y^{2 \cdot 2} = y^4$.
- For the third bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (6 \cdot 4) \bmod 17 = 24 \bmod 17 = 7$
  - $y = y^2 \bmod 17 = 4^2 \bmod 17 = 16$.
- For the fourth bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (7 \cdot 16) \bmod 17 = 112 \bmod 17 = 15$
  - $y = y^2 \bmod 17 = 16^2 \bmod 17 = 256 \bmod 17 = 1$.

So the result of $6^{13} \bmod 17$ is 15. The efficiency of the algorithm comes from the fact that it iteratively computes $y^x \bmod n$ by performing small multiplication and squaring operations thanks to the modular properties instead of performing a single large exponentiation operation.

But the timing of this algorithm can vary based on the value of the exponent x and the input y. Indeed, as seen in the above example, the algorithm performs an extra multiplication only when the bit of x is 1, which can lead to a longer execution time compared to when the bit is 0. This variation in timing is the basis for timing attacks on RSA.

2.4 Exploiting timing variations

Attackers can exploit the timing variations in the square-and-multiply algorithm to recover the private key d by measuring the time it takes for the server to perform decryption operations. Note that the attacker must know the modulus n, which is public.

First, the attacker sends multiple random ciphertexts c to the server and measures the time it takes for the server to perform the decryption operation $c^d \bmod n$.

Then for each bit of the private key d, the attacker guesses the value of the bit (0 or 1) and computes the expected timing of the decryption operation based on the guess for all the ciphertexts c.

Finally, the attacker subtracts his obtained timing measurements from the timing measurements obtained from the server and analyzes the variance of the obtained results. If the variance is low, it means that the guess for the bit is correct, while if the variance is high, it means that the guess for the bit is incorrect.

By repeating this process for all the bits of the private key d, the attacker can recover the entire private key.

2.4.1 Mathematical analysis

2.4.1.1 Probability of a correct guess

We consider N random ciphertexts $c_1, c_2, ..., c_N$ that the attacker sends to the server for decryption.

Let T_i be the time taken for the server to perform the decryption operation $c^d \bmod n$ for a given ciphertext c_i.

Let x_b be the guess for the b first bits of the private key d.

Let $t(c_i, x_b)$ be the time taken to perform the decryption operation according to the guess x_b for the b first bits of d.

If the guess x_b is correct, the timing error $T_i - t(c_i, x_b)$ will be small, indicating a good match between the observed and expected timings. We define F as the probability to observe the timing error $T_i - t(c_i, x_b)$ if the guess x_b is correct. As each timing measurement is independent, the probability of a correct guess for the b first bits of d is proportional to the product of the probabilities of F for all the ciphertexts c_i:

$$P(x_b) \propto \prod_{i=1}^{N} F(T_i - t(c_i, x_b))$$

Suppose that the $b-1$ bits of d are correct, and we want to guess the value of the b-th bit. We note x_{b_0} and x_{b_1} the guesses for the b-th bit being 0 and 1, respectively.

The probability of having x_{b_1} is:

$$P\left(x_{b_1}\right) = \frac{\prod_{i=1}^{N} F\left(T_i - t\left(c_i, x_{b_1}\right)\right)}{\prod_{i=1}^{N} F\left(T_i - t\left(c_i, x_{b_0}\right)\right) + \prod_{i=1}^{N} F\left(T_i - t\left(c_i, x_{b_1}\right)\right)}$$

If $P\left(x_{b_1}\right) > 0.5$, it means that the guess x_{b_1} is more likely to be correct than the guess x_{b_0}, and thus we can conclude that the b-th bit of d is likely to be 1.

Concretely, computing the exact distribution of F is not feasible.

2.4.1.2 Variance analysis

Instead of computing the exact distribution of F, we can analyze the variance of the timing errors for the two guesses x_{b_0} and x_{b_1}.

Suppose the private key d has l bits. The time taken for the decryption operation can be denoted as $T_i = \sum_{k=1}^{l} t_k + \varepsilon$ with t_k the time taken for the k-th step of the algorithm and ε representing the timing noise caused by various factors such as system load, network latency, or other sources of randomness.

Suppose that x_{b-1} is known to be correct. The attacker can compute the time taken for each step of the first $b-1$ bits as $\sum_{k=1}^{b-1} t_k$.

The time difference is then:

$$T_i - t(c_i, x_{b-1}) = \sum_{k}^{l} t_k + \varepsilon - \sum_{k}^{b-1} t_k = \sum_{k=b}^{l} t_k + \varepsilon$$

If the guess x_b is correct, the time difference will be:

$$T_i - t(c_i, x_b) = \sum_{k=b+1}^{l} t_k + \varepsilon$$

And if the guess x_b is incorrect, the time difference will be:

$$T_i - t(c_i, x_b) = \sum_{k}^{l} t_k + \varepsilon - \left(\sum_{k}^{b-1} t_k + t_{b_{\text{incorrect}}} \right) = \sum_{k=b+1}^{l} t_k + \varepsilon + \left(t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}} \right)$$

.

We can constate that the time difference for the correct guess x_b is smaller than the time difference for the incorrect guess x_b by a factor of $t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}}$, meaning that the variance of the time differences for the correct guess x_b will be smaller than the variance of the time differences for the incorrect guess x_b.

So by analyzing the variance of the time differences for the two guesses x_{b_0} and x_{b_1}, the attacker can determine which guess is more likely to be correct, and thus recover the bits of the private key d one by one.

2.4.1.3 Impact of previous errors on the variance

Now, suppose that the c-th bit of d is incorrect for some $c < b-1$. The time difference for the correct guess x_b will be:

$$T_i - t(c_i, x_b) = \sum_{k}^{l} t_k + \varepsilon - \left(\sum_{k}^{c-1} t_k + \sum_{k=c}^{b} t_k \right) = \sum_{k=b+1}^{l} t_k + \varepsilon + \left(\sum_{k=c}^{b} \left(t_{k_{\text{correct}}} - t_{k_{\text{incorrect}}} \right) \right)$$

We have t_k which starts to be different for the correct and incorrect guesses for all the bits from c to b since the intermediate steps of the algorithm will be executed differently due to the error in the guess for the c-th bit, leading to a different t_k for all the bits from c to b even if the guess for the b-th bit is correct.

So when the guess x_b has some previous errors, the variance of the correct guess will start to be similar to the variance of the incorrect guess.

This property can be exploited to detect errors in the guess for the b-th bit. Indeed, attacker can keep track of the variance of the time differences and come back to a previous guess if

the variance starts to be too similar for both the correct and incorrect guesses, indicating that there might be an error in the previous bits of the guess.

2.5 Code implementation

The code implementation was done first in Python, and then in Rust and C++ to try to reduce the noise in the timing measurements and thus increase the chances of success of the attack.

Indeed, the python implementation was not successful at all in recovering the private key due to the high noise caused by the Python interpreter and its optimizations, which totally masked the timing variations caused by the square-and-multiply algorithm.

2.5.1 Sources of noise

There are several sources of noise that can affect the timing measurements in programming languages, especially in high-level languages like Python.

2.5.1.1 CPU cache effects

CPU cache effects that can cause variability in the timing measurements based on the memory access patterns of the algorithm.

First, there are different levels of CPU cache (L1, L2, L3) that can store recently accessed data and instructions.

These caches are designed to speed up access to frequently used data and instructions, but they can also introduce variability in the timing measurements. Indeed, the cache L1 is the fastest but also the smallest, while the cache L3 is the slowest but also the largest, meaning that if the data or instructions needed for the algorithm are in the cache L1, the timing measurements will be faster compared to when they are in the cache L3 or not in the cache at all.

If the data or instructions needed are not in the cache, the CPU triggers a cache miss, which means that it has to fetch the data from the main memory, leading to a significant delay in the execution of the algorithm and thus in the timing measurements.

These cache effects represent a significant source of noise in timing measurements.

2.5.1.2 Branch prediction

Branch prediction is a technique used by modern CPUs to improve performance by guessing the outcome of conditional statements and executing instructions based on those guesses. When the CPU encounters a conditional statement (e.g., an if statement), it makes a guess about which branch of the code will be executed next based on past behavior and patterns. This allows the CPU to continue executing instructions without waiting for the outcome of the conditional statement, which can improve performance.

So if the CPU correctly predicts the branch, it can continue executing instructions without interruption, but if the CPU incorrectly predicts the branch, it has to discard the incorrectly executed instructions and fetch the correct instructions, leading to a significant delay in the execution of the algorithm and thus in the timing measurements.

That's why branch prediction can cause variability in the timing measurements based on the input values and the internal state of the algorithm, as different inputs can lead to different execution paths and thus different branch predictions.

2.5.1.3 Garbage collection

Garbage collection is a form of automatic memory management that is used in many programming languages, including Python. It is responsible for automatically freeing up memory that is no longer in use by the program, which can help prevent memory leaks and improve performance.

However, the garbage collector can be triggered at any time during the execution of the program. When the garbage collector runs, it can cause significant delays in the execution of the program: it has to pause the execution of the program, scan the memory for objects that are no longer in use, and free up the memory occupied by those objects.

This can lead to significant variability in the timing measurements since the garbage collector can be triggered at different times during the execution of the algorithm, leading to different timing measurements for the same operations.

2.5.1.4 Multiplication optimizations

The usage of modern programming languages introduces various optimizations that can affect the timing measurements, such as the optimization of multiplication operations.

The multiplication of large integers can be optimized using different algorithms depending on the size of the numbers. For small integers, the standard multiplication algorithm is used whereas for larger integers, more efficient algorithms such as Karatsuba or Toom-Cook can be used.

These optimizations can lead to different timing measurements for the same operations based on the size of the numbers being multiplied.

2.5.1.5 Other optimizations

There are also other optimizations that can be introduced by the programming language or the compiler, such as loop unrolling, instruction reordering, or just-in-time compilation, which can further introduce variability in the timing measurements.

So the sources of noise in timing measurements can be quite significant, especially in high-level programming languages like Python, and they can totally mask the timing variations caused by the square-and-multiply algorithm.

2.5.2 Reducing the impact of noise

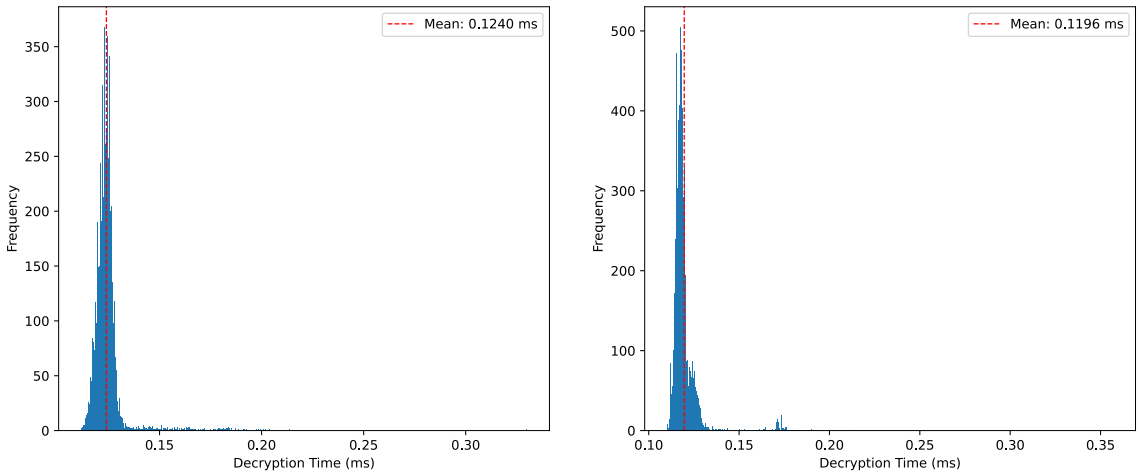
There are several techniques that can be used to try to reduce the impact of these sources of noise in timing measurements.

2.5.2.1 Averaging timing measurements

The first approach can consist to perform a large number of timing measurements for each sample and then use statistical analysis to try to extract the signal from the noise, such as computing the mean or the median of the timing measurements for each sample.

However, as shown on Figure 1, there are some samples for which the timing measurements are significantly higher than the others, which can be caused by the garbage collector or other sources of noise described in Section 2.5.1, and these outliers can significantly affect the mean and thus the analysis of the timing measurements.

Indeed, on the Figure 1, we can see that the mean of the timing measurements of the key 0 is higher than the mean of the timing measurements of the key 1, which is not expected since the key 0 should be faster than the key 1 due to the extra multiplication performed when the bit of the key is 1.



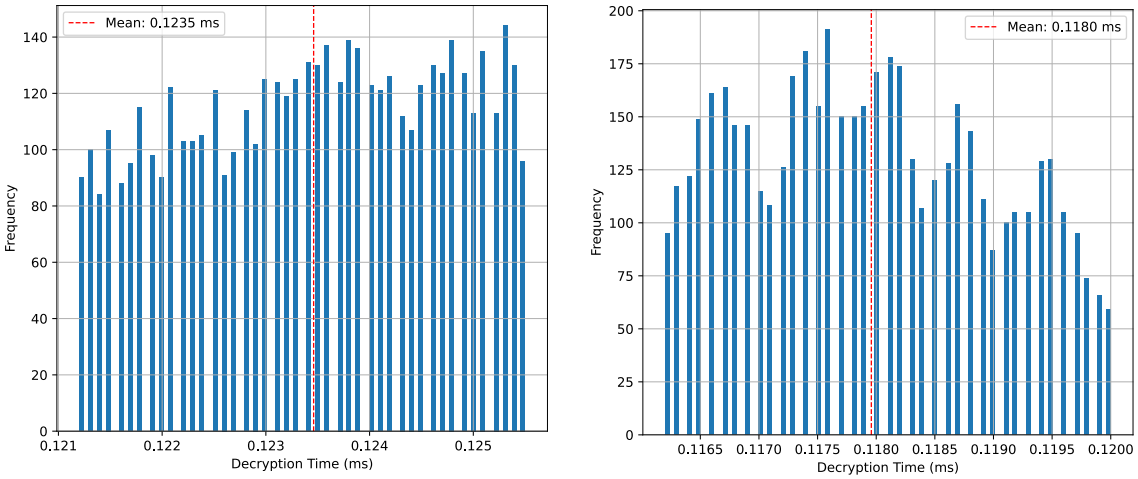
(a) Key 0 (b) Key 1

Figure 1: 10000 timing measurements of the decryption operation on a single ciphertext for two different private keys (0 and 1)

A way to mitigate the impact of these outliers is to filter them out by removing the timing measurements that are too far from the center of the distribution. This can be done by removing the timing measurements that are bellow or above a certain percentile of the distribution, such as bellow 25% and above 75%, which can help to reduce the impact of outliers on the analysis of the timing measurements by keeping only the most representative samples.

However, as shown on Figure 2, even after removing the outliers, the average timing measurements for the key 0 are still higher than the average timing measurements for the key 1, which is not expected since the key 0 should be faster than the key 1 due to the extra multiplication performed when the bit of the key is 1.

This can be explained by the fact that the sources of noise described in Section 2.5.1 are still present in the timing measurements, making the averaging of the timing measurements not sufficient to extract the signal from the noise.



(a) Key 0

(b) Key 1

Figure 2: Removing outliers from the timing measurements by keeping only the samples between 25% and 75%

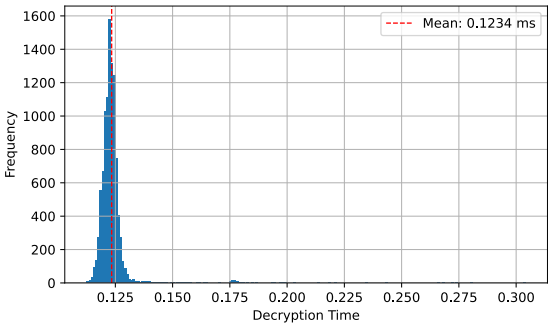
2.5.2.2 Warm-up and disabling garbage collection

As making a large number of timing measurements is not sufficient to extract the signal from the noise, another approach can consist to try to reduce the sources of noise in the timing measurements.

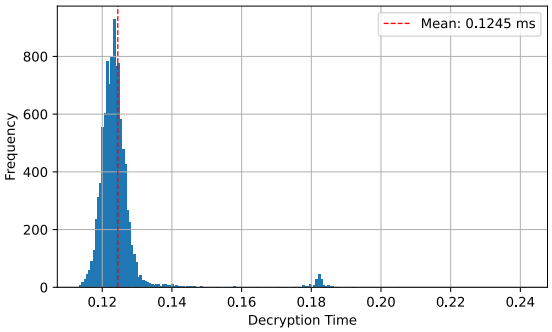
As mentioned in Section 2.5.1, the garbage collector is a significant source of noise in the timing measurements, so one way to reduce the noise is to disable the garbage collector during the timing measurements.

This can be done using the `gc` module in Python, which provides functions to enable and disable the garbage collector.

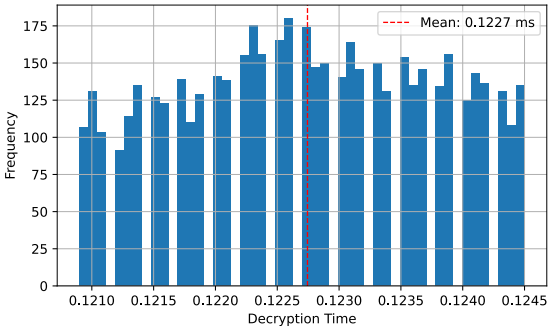
The obtained results shown on Figure 3 are much better than the previous results shown on Figure 1 and Figure 2, as we can see that the average timing measurements for the key 0 are now lower than the average timing measurements for the key 1 as expected.



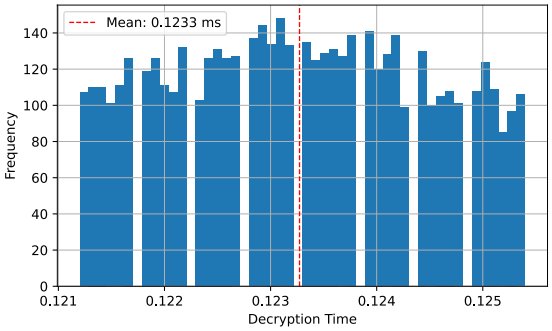
(a) Hypothesis h0: the last bit is 0



(b) Hypothesis h1: the last bit is 1



(c) Hypothesis h0: the last bit is 0 (without outliers)



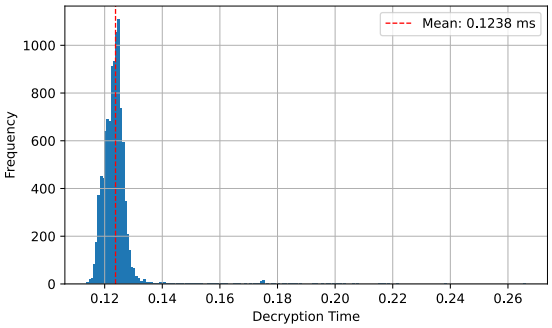
(d) Hypothesis h1: the last bit is 1 (without outliers)

Figure 3: Timing measurements with garbage collection disabled

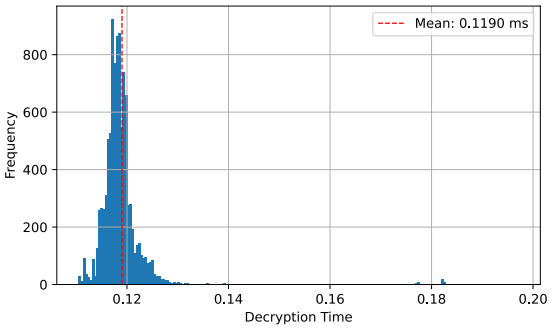
Now, the problem is that the two measurements for the key 0 and the key 1 are still quite close to each other. To try to further reduce the noise in the timing measurements and improve the distinction between the two keys, we can add a warm-up phase before the timing measurements.

The warm-up phase consists in performing a certain number of decryption operations before the timing measurements. In this way, the CPU can load the necessary data and instructions into the cache, and the branch predictor can learn the patterns of the algorithm, which can help to reduce the variance of the timing measurements. The warm-up phase is a common technique used in performance benchmarking to ensure that the measurements are more stable and representative of the actual performance of the algorithm.

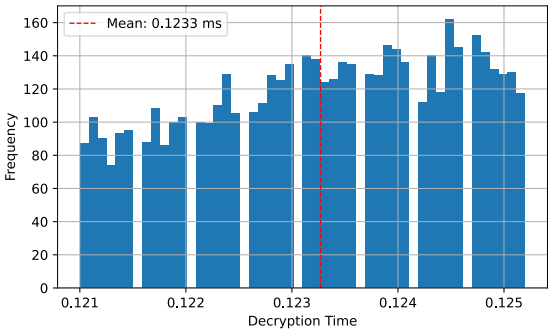
The obtained results shown on Figure 4 have a much better distinction between the two keys compared to the previous results shown on Figure 3.



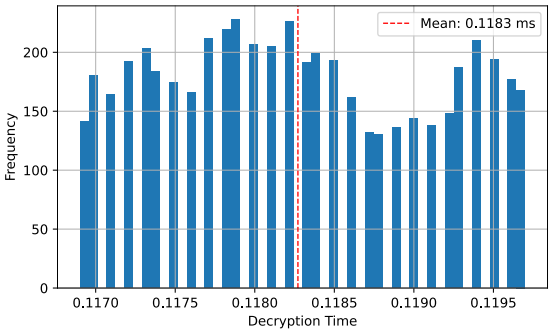
(a) Hypothesis h0: the last bit is 0



(b) Hypothesis h1: the last bit is 1



(c) Hypothesis h0: the last bit is 0 (without outliers)



(d) Hypothesis h1: the last bit is 1 (without outliers)

Figure 4: Timing measurements with garbage collection disabled and warm-up phase

Personally, I have tried to implement the attack in Rust and C++ to try to reduce the noise caused by the Python interpreter and its optimizations, but the attack was still not successful at all in recovering the private key due to the high noise caused by the CPU cache effects, branch prediction, and other optimizations.

I have tried to implement a simple version of the timing attack on RSA in Python, based on the square-and-multiply algorithm for modular exponentiation.

The code contains two versions of the attack:

- A simple version that does not take into account the impact of previous errors on the variance, which can lead to a higher number of errors in the recovered private key.
- An improved version that manage a fixed size list of the best guesses for the private key, and that compute each time the new guess on all the list of best guesses, allowing to avoid the impact of previous errors on the variance and thus reduce the number of errors in the recovered private key. Note that if the list of best guesses contains only guesses with some errors, the attack can still fail, but in practice, it allows to significantly reduce the number of errors in the recovered private key.

However, the attack was not successful at all in recovering the private key, even with the improved version. Indeed, Python introduces a lot of noise in the timing measurements, such as:

- The garbage collector, which can be triggered at any time and can cause significant delays in the execution of the code.

- The optimization which can change the algorithm used for multiplication depending on the size of the numbers, leading to different timing measurements for the same operations.
- The branch prediction that can cause variability in the timing measurements based on the input values and the internal state of the algorithm.
- CPU cache effects that can cause variability in the timing measurements based on the memory access patterns of the algorithm.
- Other CPU and Python optimizations that can introduce variability in the timing measurements.

These sources of noise have totally masked the timing variations caused by the square-and-multiply algorithm, making it impossible to recover the private key using the timing attack, even with a large number of repetitions of timing measurements (up to 100k messages and 10000 repetitions in Rust), the disabling of the garbage collector and the addition of a delay before each timing measurement to try to reduce the impact of the optimizations and branch prediction.

2.6 Fermat's factorization method

The private key d in RSA is computed based on the prime factors p and q of the modulus n. So another way to recover the private key d is to factor the modulus n into its prime factors p and q, and then compute d using the formula $d \equiv e^{-1} \bmod \varphi(n)$ with $\varphi(n) = (p-1)(q-1)$ the Euler's totient function. The inverse of e modulo $\varphi(n)$ can be computed using the Extended Euclidean Algorithm, which is an efficient method for computing the greatest common divisor of two integers and their multiplicative inverse.

But factoring large integers is a computationally hard problem, and the security of RSA relies on this fact.

However, if p and q are close to each other, meaning that the difference between p and q is small, then it becomes easier to factor n using Fermat's factorization method.

Fermat's factorization method relies on the fact that n is a product of two odd primes p and q. As p and q are odd, there always exist an integer number which is at the middle of p and q, which is $x = \frac{p+q}{2}$. Then, we consider y as the distance between x and p (or q):

$$y = x - p = q - x$$

So we have:

- $p = x - y$
- $q = x + y$

If we express n in terms of x and y, we get:

$$n = p \times q = (x-y)(x+y) = x^2 - y^2$$

From this equation, we can deduce that $y^2 = x^2 - n$.

The Fermat's factorization method consists in finding the smallest integer x such that $y^2 = x^2 - n$ is a perfect square, meaning that y is an integer.

Once we find such an integer x, we can compute y as $y = \sqrt{x^2 - n}$, and then we can obtain the prime factors p and q as described above.

So the algorithm can be summarized as follows:

1. Compute $x = \lceil \sqrt{n} \rceil$.
2. Compute $y^2 = x^2 - n$.
3. If y^2 is a perfect square, then $y = \sqrt{y^2}$ and we can compute $p = x - y$ and $q = x + y$.
4. Otherwise, increment x and repeat from step 2.